



Volatile y Objetos Atómicos

Volatile



`volatile` es un modificador de variable que afecta a **cómo la JVM gestiona la visibilidad y el orden de memoria** de esa variable (es propio de Java).

Explicación

Cuando declaramos una variable como `volatile`, le indicamos al compilador y a la JVM que:

- **su valor puede ser modificado por otros hilos,**
- y por tanto **no debe cachearse localmente** en registros o cachés de hilo.

```
private static volatile boolean running = true;
```

Esto garantiza dos cosas fundamentales:

1. Visibilidad:

Todo hilo que lea la variable ve siempre **el último valor escrito** por cualquier otro hilo. Nos garantiza la consistencia en memoria para operaciones de lectura/escritura.

2. Orden de memoria (happens-before):

Las lecturas/escrituras sobre la variable `volatile` no pueden reordenarse alrededor suyo.

Es decir, los cambios realizados en una variable volatile son SIEMPRE VISIBLES para el resto de hilos.

¿Qué NO garantiza volatile?

Aunque la lectura/escritura de la variable se vuelve **atómica**, esto solo aplica a operaciones simples:

- lectura → atómica
- escritura → atómica

Las **operaciones compuestas** NO son atómicas:

- `x++`
- `x += y`
- `if (flag) { ... }`
- `x = x + 1` (3 instrucciones máquina)

Por tanto, aunque la variable sea `volatile`, estas operaciones seguirán siendo vulnerables a entrelazados.

Ejemplo de error:

```
public class VolatileIncrementDemo {
    private static volatile int contador = 0;

    public static void main(String[] args) throws InterruptedException {
        Runnable tarea = () → {
            for (int i = 0; i < 10000; i++) {
                contador++; // NO es atómico, aunque sea volatile
            }
        };

        Thread t1 = new Thread(tarea);
        Thread t2 = new Thread(tarea);

        t1.start();
        t2.start();

        t1.join();
        t2.join();
    }
}
```

```
System.out.println("Esperado: 20000");
System.out.println("Real:   " + contador); // casi siempre < 20000
}
}
```

`contador++` ejecuta internamente:

1. cargar valor
2. incrementarlo
3. almacenarlo

→ en presencia de hilos concurrentes, pasos 1 y 2 **se entrelazan**, generando pérdidas.

¿Para qué sirve volatile?

- `volatile` = **visibilidad + orden**, NO **exclusión mutua**.
- `volatile` NO soluciona condiciones de carrera.
- Las operaciones compuestas requieren **sección crítica** (`synchronized` , Lock, monitores...).
- `volatile` sirve para **flags de control**: parar hilos, notificar estados, etc.

Recordar también que en Java todos los tipos primitivos tienen ya las operaciones de lectura y escritura de forma atómica. Volatile solo sería realmente necesario para tipos NO PRIMITIVOS como long, double, u otros.

Además, también recordar siempre que este modificador NO GARANTIZA la atomicidad de operaciones compuestas.

Si queremos eso, mejor usar objetos atómicos.

Objetos Atómicos



Los objetos atómicos son tipos de la API de alto nivel diseñados para realizar **operaciones compuestas atómicas** sobre una variable compartida

Solo están disponibles en la API de Alto Nivel.

A diferencia de `volatile`, estos tipos:

- ✓ Garantizan visibilidad (por defecto, todo objeto atómico será `volatile`).
- ✓ Garantizan atomicidad de operaciones del API (pero no de todos los métodos).
- ✗ NO garantizan atomicidad de *secuencias* de operaciones
- ✗ NO reemplazan completamente a la sincronización clásica.

Relación entre `volatile` y objetos atómicos

Todo objeto atómico será `volatile`, pero no toda variable `volatile` será un objeto atómico.

Al igual que pasaba con las variables `volatile`, nos aseguramos que las operaciones de lectura/escritura son ATÓMICAS.

¿Cómo funcionan?

Internamente:

- almacenan el valor en un campo `volatile`,
- implementan operaciones usando instrucciones atómicas del procesador (CAS),
- y exponen estas operaciones como métodos seguros.

Ejemplo de uso

```
import java.util.concurrent.atomic.AtomicInteger;

public class AtomicIncrementDemo {
    private static final AtomicInteger contador = new AtomicInteger(0);

    public static void main(String[] args) throws InterruptedException {
        Runnable tarea = () -> {
            for (int i = 0; i < 10000; i++) {
                contador.getAndIncrement(); // operación compuesta atómica
            }
        };
    }
}
```

```

Thread t1 = new Thread(tarea);
Thread t2 = new Thread(tarea);

t1.start();
t2.start();

t1.join();
t2.join();

System.out.println("Esperado: 20000");
System.out.println("Real:   " + contador.get());
}
}

```

→ `getAndIncrement()` es **atómico** por definición del API.

¿Nos ofrecen atomicidad o no?

No debemos usar los métodos que nos ofrecen estas clases así como así, ya que puede ser que alguno no sea atómico.

Para saberlo, hay que mirar la API y ver cómo se comporta.

En caso de que dicho método SÍ SEA ATÓMICO, la operación será atómica.

Ahora, debemos tener cuidado, porque el uso conjunto de sus métodos NO ES ATÓMICO.

Por ejemplo:

```

if (contador.get() < 10) {
    contador.incrementAndGet(); // NO es atómico como conjunto
}

```

Entre el `get()` y el `incrementAndGet()` pueden colarse otros hilos.